

Prince Sultan University

College of Computer & Information Sciences

Information Systems Department

IS499 – Senior Project

D6 – Software User Guide

Mirsad (مرصاد)

A Runtime Security Firewall for AI Agents Against Prompt-Injection Attacks

Team member	Student ID	Department
Mohammed Yasser Alanssari	221110935	IS (CYS)
Bader Alshuraie	221110016	SE
Bader Abdullah Abbod Alyamini	221110197	IS (CYS)
Abdulmalik bin Saedan	221110958	IS (CYS)
Abdullah Naif Abdullah Al-Orabi	222110432	SE (AI & DS)

Supervisor	Dr. Omaia Al-Omari
Semester & Academic Year	Senior Project I & II – Academic Year 2026–2027
Submission Date	_____
Supervisor's Signature	_____
Approval Date	_____

Table of Contents

Table of Contents i
List of Figures iii
Mirsad Software User Guide 1
E.1 Overview and System Requirements 1
E.2 Installation 2
E.2.1 Python virtual environment 2
E.2.2 Docker Compose with PostgreSQL 2
E.3 Configuration 3
E.4 First Run and Demonstration 4
E.4.1 Initialising and starting the server 4
E.4.2 Local model for the demo agent 4
E.4.3 Running a demonstration 5
E.5 Administration Dashboard 5
E.5.1 Signing in and navigation 6
E.5.2 Overview 6
E.5.3 Activity and audit log 6
E.5.4 Approvals 7
E.5.5 Policies, agents and users 8
E.5.6 Tools, detector playground, demo console and settings 9
E.6 Approver App 11
E.6.1 Installing on Android and iOS 11
E.6.2 Reviewing and deciding 12
E.6.3 Notifications 13
E.6.4 Arabic interface 14
E.7 Writing and Activating Policies 14
E.7.1 Policy structure and an example 14
E.7.2 Validating 15
E.7.3 Activating, monitor mode and rollback 16
E.8 Connecting Agents 16

E.8.1 Before connecting 16
E.8.2 MCP hosts: stdio proxy and HTTP 17
E.8.3 OpenAI-compatible agents 17
E.8.4 Python SDK 17
E.9 Command-Line Reference 18
E.10 Audit Verification, Replay and Export 18
E.11 Training the Detector and Running the Benchmark 19
E.11.1 Detector 19
E.11.2 Benchmark 20
E.12 Troubleshooting and FAQ 20
E.13 Security Recommendations for Operators 21
References 23

List of Figures

Figure 1: Overview page of the dashboard 6
Figure 2: Activity and audit log with the detail drawer of an approval decision..... 7
Figure 3: Approvals page with two pending requests and the decision history..... 8
Figure 4: Policies page with the YAML editor..... 9
Figure 5: Agents page with the demo agent created by mirsad init --demo 9
Figure 6: Detector playground scanning an Arabic business e-mail 10
Figure 7: Demo console with the protection switch, scenario selector and attacker inbox 11
Figure 8: Approver app: pending requests..... 12
Figure 9: Approver app: request detail with the Approve and Deny buttons 13
Figure 10: Approver app: request detail in Arabic 14

Mirsad Software User Guide

This guide is deliverable D6 of the Mirsad senior project. It explains how to install, configure, operate and integrate Mirsad, for administrators, approvers and agent developers.

This document is an extract of the Mirsad Final Report, where its content appears as Appendix E. Section, figure and table numbers, and references to other sections, chapters and appendices, follow that report.

This appendix is the Software User Guide of Mirsad 1.0.0 (IS499 deliverable D6). It explains how to install, configure, operate and integrate the system; Table 1 shows which sections each reader needs. Commands are run from the project root, and `mirsad` is equivalent to `python -m mirsad.cli`. Measured values are read from `reports/results_summary.json` when the document is built; "[pending]" marks a value that has not been produced yet.

Table 1: Reading guide by audience

Reader	Typical tasks	Sections
Administrators and IT staff	Install, configure, manage users and agents, write policies, harden the deployment	E.1–E.5, E.7, E.9, E.12, E.13
Approvers	Install the phone app, receive alerts, approve or deny paused actions	E.5.4, E.6
Auditors	Search activity, verify the hash chain, replay decisions, export evidence	E.5.3, E.10
Agent developers	Connect agents through MCP, the OpenAI-compatible proxy or the SDK; train the detector; run the benchmark	E.8, E.11, E.12

E.1 Overview and System Requirements

Mirsad is one Python service (FastAPI on Uvicorn) placed between AI agents and their tools. For each tool call it evaluates the active YAML policy, rate limits and the session's taint state, then allows, denies or pauses the call for a person. After a tool runs, Mirsad scans the result for instructions aimed at the agent, sanitises or blocks them, and marks the content as untrusted data. Every step is written to an audit log whose events are chained with an HMAC [1]. The same process serves the agent endpoints (`/mcp` for MCP [2], `/v1/chat/completions`, and the Guard API under `/v1/`), the dashboard at `/`, the approver app at `/approve` and API documentation at `/docs`.

Table 2: System requirements

Item	Requirement	Notes

Item	Requirement	Notes
Operating system	macOS or Linux with Bash	Tests and measurements ran on macOS (Apple Silicon); Windows was not tested
Python	3.11 or later (3.12 recommended)	
Node.js	20.19 or later	Only to rebuild the web app; a build ships in `src/mirsad/static/`
Database	SQLite (default) or PostgreSQL	Compose uses PostgreSQL 16
Hardware	CPU only	No GPU needed
Demo model (optional)	Ollama serving `qwen2.5:7b`	Demo Console, demo agent, LLM benchmark
Approver phone	Android (Chrome, Edge, Samsung Internet) or iOS/iPadOS 16.4+ (Safari)	Server reachable over HTTPS

On the development machine the release passed 1,281 of 1,281 Python tests and 194 of 194 web tests (94.4% statement coverage). With the gateway in a separate process, Mirsad added a median of 2.7 ms (p95 6.0 ms) to a sequential tool call over HTTP, with the detector in `rules_only` mode.

E.2 Installation

E.2.1 Python virtual environment

```
cd mirsad                 # repository clone or release archive
./scripts/setup.sh        # or: make setup
source .venv/bin/activate
mirsad --version          # mirsad, version 1.0.0
```

The script checks for Python 3.11 or later (`PYTHON_BIN` selects an interpreter), creates `.venv`, installs Mirsad in editable mode with the development extras, and runs `npm ci` in `web/` when `npm` exists. `make web` rebuilds the dashboard and approver app into `src/mirsad/static/`; without a build, `/` returns a short JSON note.

E.2.2 Docker Compose with PostgreSQL

The two-stage `Dockerfile` builds the web app with Node 22, then installs Mirsad with the PostgreSQL driver on `python:3.12-slim`, running as the unprivileged user `mirsad` with data in the `/data` volume, port 8700 and a `/healthz` health check. `docker-compose.yml` defines `mirsad`, `db` (`postgres:16-alpine`) and an optional `ollama` service in the `demo` profile.

> **Status.** The Dockerfile and Compose file were written but could not be executed on the development machine because Docker was not available there; treat them as reviewed but untested. PostgreSQL compatibility was verified separately: `tests/integration/test_postgres.py` starts an embedded PostgreSQL server (`pgserver` package) and runs the audit chain, tamper detection, authentication and approvals against it. These tests passed in the recorded run (`reports/tests/junit.xml`).

1. Copy `.env.example` to `.env` and replace every value: `MIRSAD_SECRET_KEY`, `MIRSAD_ADMIN_PASSWORD` (creates the first administrator at first start), `POSTGRES_PASSWORD` and `MIRSAD_ENABLE_DEMO` (`false` in production).
2. Add the two lines below to the `environment` of the `mirsad` service.
3. Run `docker compose up -d --build`; check with `docker compose ps` and `docker compose logs mirsad`.
4. Sign in at `http://<host>:8700/` as `admin` and create approvers on the Users page or with `docker compose exec mirsad mirsad user create <name> --role approver`.
5. For the demo: `MIRSAD_ENABLE_DEMO=true`, `docker compose --profile demo up -d`, `docker compose exec ollama ollama pull qwen2.5:7b`.

```
MIRSAD_POLICY_PATH: /app/configs/policies/default.yaml
MIRSAD_UPSTREAMS_PATH: /app/configs/upstreams.yaml
```

The defaults of these two settings are computed relative to the source tree. In the image, which installs Mirsad as a regular package, they point inside the Python installation, so the server would start with the built-in deny-all policy and no tools. This follows from `src/mirsad/config.py` and was not observed in a running container; it applies to any non-editable installation.

E.3 Configuration

Settings come from environment variables prefixed `MIRSAD_` or from a `.env` file in the start directory (environment variables win) and are read at start-up. Policies are stored in the database instead (E.7).

Table 3: Environment variables and defaults

Variable	Default	Purpose
`MIRSAD_HOST`, `MIRSAD_PORT`	`127.0.0.1`, `8700`	Bind address and port
`MIRSAD_DATA_DIR`	`.mirsad` (Docker: `/data`)	SQLite file, `secret.key`, `credentials.txt`
`MIRSAD_DATABASE_URL`	SQLite in the data directory	SQLAlchemy URL, e.g. `postgresql+psycopg://user:pw@host/mirsad`
`MIRSAD_SECRET_KEY`	generated once into `secret.key`	Signs sign-in tokens; the audit HMAC key is derived from it
`MIRSAD_POLICY_PATH`	`configs/policies/default.yaml`	Initial policy when the database has none
`MIRSAD_UPSTREAMS_PATH`	`configs/upstreams.yaml`	Tool servers that Mirsad protects
`MIRSAD_DETECTOR_MODEL_DIR`	`src/mirsad/detector/model`	Trained detector; rules-only mode when absent
`MIRSAD_TOKEN_TTL_HOURS`	`12`	Sign-in lifetime
`MIRSAD_APPROVAL_TIMEOUT_S`	`120`	Time to decide; an expired request is denied

Variable	Default	Purpose
`MIRSAD_MAX_RESULT_CHARS`	`200000`	Longer tool results are truncated before scanning
`MIRSAD_AUDIT_PAYLOAD_MAX_CHARS`	`20000`	Size limit of one audit payload
`MIRSAD_ENABLE_DEMO`	`false`	Enables the Demo Console endpoints
`MIRSAD_LLM_BASE_URL`	`http://127.0.0.1:11434/v1`	LLM for the OpenAI-compatible proxy and demo agent
`MIRSAD_LLM_MODEL`, `MIRSAD_LLM_API_KEY`	`qwen2.5:7b`, `ollama`	Default model and upstream credential
`MIRSAD_CORS_ORIGINS`	the Vite development server (port 5173)	Allowed browser origins (JSON list)
`MIRSAD_VAPID_PUBLIC_KEY`, `MIRSAD_VAPID_PRIVATE_KEY`	empty	Enable Web Push (E.6.3)
`MIRSAD_VAPID_SUBJECT`	`mailto:security@najd-trading.example`	Contact sent to push services
`MIRSAD_LOG_LEVEL`	`INFO`	Log level (standard error)
`MIRSAD_ADMIN_PASSWORD`, `MIRSAD_ADMIN_USERNAME`	unset, `admin`	Creates the first administrator when no user exists
`MIRSAD_NEW_USER_PASSWORD`	unset	Password for `mirsad user create` without a prompt
`MIRSAD_AGENT_KEY`, `MIRSAD_URL`	unset, `http://127.0.0.1:8700`	Agent side: `mirsad mcp-proxy` and `MirsadClient.from_env()`

`POSTGRES_PASSWORD` is used by the Compose file and `MIRSAD_API` by the web development server.

E.4 First Run and Demonstration

E.4.1 Initialising and starting the server

```
mirsad init --demo     # database, first policy version, users, demo agent
mirsad serve --demo    # dashboard /, approver app /approve, API docs /docs
```

`init` stores the default policy as the first version, creates the users `admin`, `approver` and `auditor` and, with `--demo`, the agent `demo-agent` (policy agent `email-crm-assistant`). It prints only the path of `.mirsad/credentials.txt` (mode 600), which holds the generated passwords and the agent key; move them to a password manager and delete the file. Re-running `init` creates only what is missing. `make demo` runs both steps through `scripts/run_demo.sh`. Then sign in at `http://127.0.0.1:8700/` as `admin`, and open `/approve` as `approver` in a second window.

E.4.2 Local model for the demo agent

The demo agent uses Ollama's OpenAI-compatible endpoint [3] with Qwen2.5-7B-Instruct [4]. Run `ollama pull qwen2.5:7b`; `curl -s http://127.0.0.1:11434/v1/models`

should list it. Without the model the Demo Console returns error 502; nothing else depends on it.

E.4.3 Running a demonstration

1. In **Demo console**, choose a scenario, for example the invoice e-mail whose body hides an instruction to send the customer list outside. **Load scenario** resets the simulated Najd Trading Co. workspace, plants the content and fills in a harmless user request.
2. Turn **Mirsad protection** off and send. Whether the agent follows the planted instruction depends on the model; anything that leaves the organisation appears in the **Attacker inbox**.
3. **Reset demo**, load the scenario again with protection on and send. The tool-call timeline shows each decision and its reason; paused calls reach the Approvals page and the approver app.

The command line runs the same scenarios and prints an outcome computed from the workspace side effects: `python -m demo.agent --list`, then `python -m demo.agent --protected --scenario invoice-exfil --lang en` (or `--unprotected`; `--approve` auto-approves escalations).

E.5 Administration Dashboard

The sidebar shows only the pages the signed-in role may use (Table 4); the API enforces the same roles. The screenshots were captured by `scripts/capture_screenshots.py`, which starts a demo server, generates activity through the real API and photographs each page in Chrome. That server used a 900-second approval timeout, so countdowns show about 14–15 minutes instead of the default 120 seconds.

Table 4: Dashboard pages and roles

Page	Roles	Main actions
Overview	all	Key figures, charts, audit chain status
Activity & audit	admin, auditor	Search, inspect, replay, verify, export
Approvals	admin, approver	Live queue, decisions with notes, history
Policies	admin (auditor read-only)	Edit, validate, compare, activate, roll back
Agents, Users	admin	Identities, keys, accounts and roles
Tools	admin, auditor	Discovered tools and their policy treatment
Detector playground	all	Scan Arabic, English or mixed text
Demo console	all (demo mode)	Chat with the demo agent
Settings	all	Theme, language, digits, desktop alerts

E.5.1 Signing in and navigation

After 5 failed sign-ins for one username and address (or 30 from one address) within 5 minutes the page reports "too many attempts" until the window passes. The sign-in lasts while the browser tab is open. The top bar holds the live indicator, the English/العربية toggle, the theme, the account and sign-out.

E.5.2 Overview

The Overview (Figure 1) summarises the last hour, day or week: tool calls, allowed, denied, sent for approval, waiting, injections detected, results sanitised or blocked, and the median and 95th-percentile gateway overhead (excluding human waiting). Charts show decisions over time, per tool and per defence layer, each with a **Show table** view. **Verify now** re-checks the audit chain.

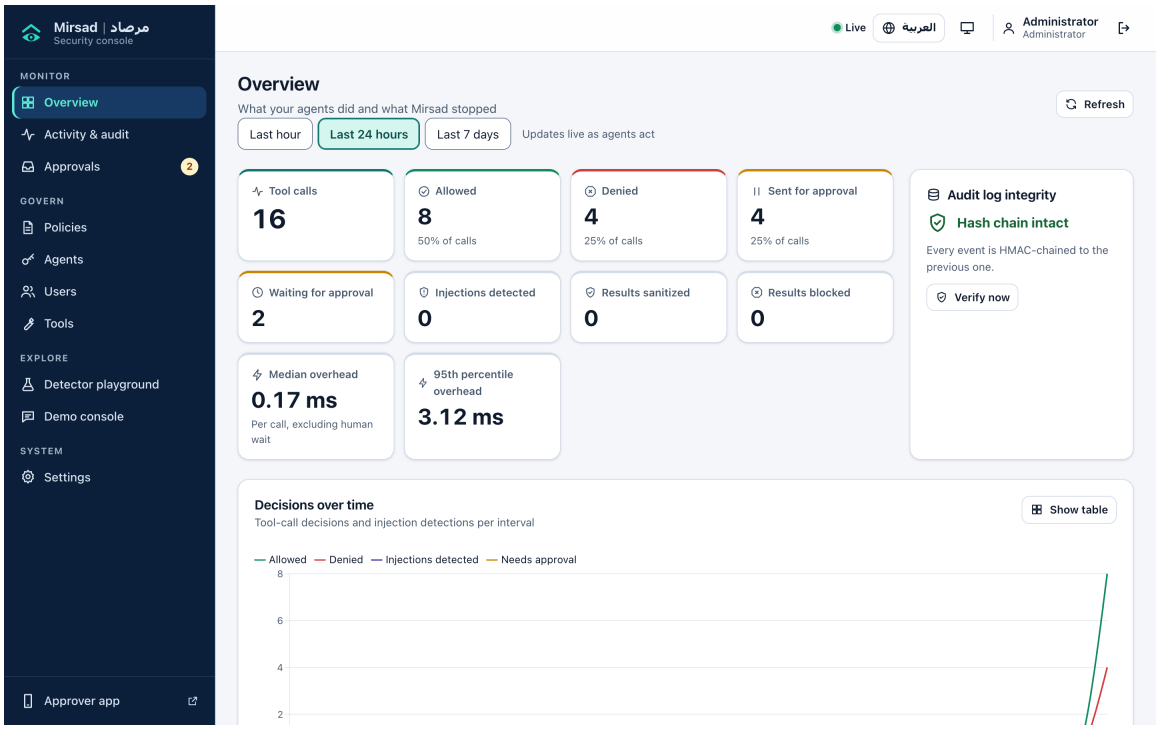


Figure 1: Overview page of the dashboard

E.5.3 Activity and audit log

A paged list of all audit events, filtered by text, agent, tool, outcome, event type, session, call or time; filters stay in the address so a view can be shared. A row opens the detail drawer (Figure 2): summary, reasons with bilingual explanations and evidence, detector spans for inspections, taint provenance (which earlier call and field a value came from), arguments, hashes and the raw event, with **Verify chain**, **Export event**, **All events of this call/session** and, for decisions and inspections, **Replay** (E.10).

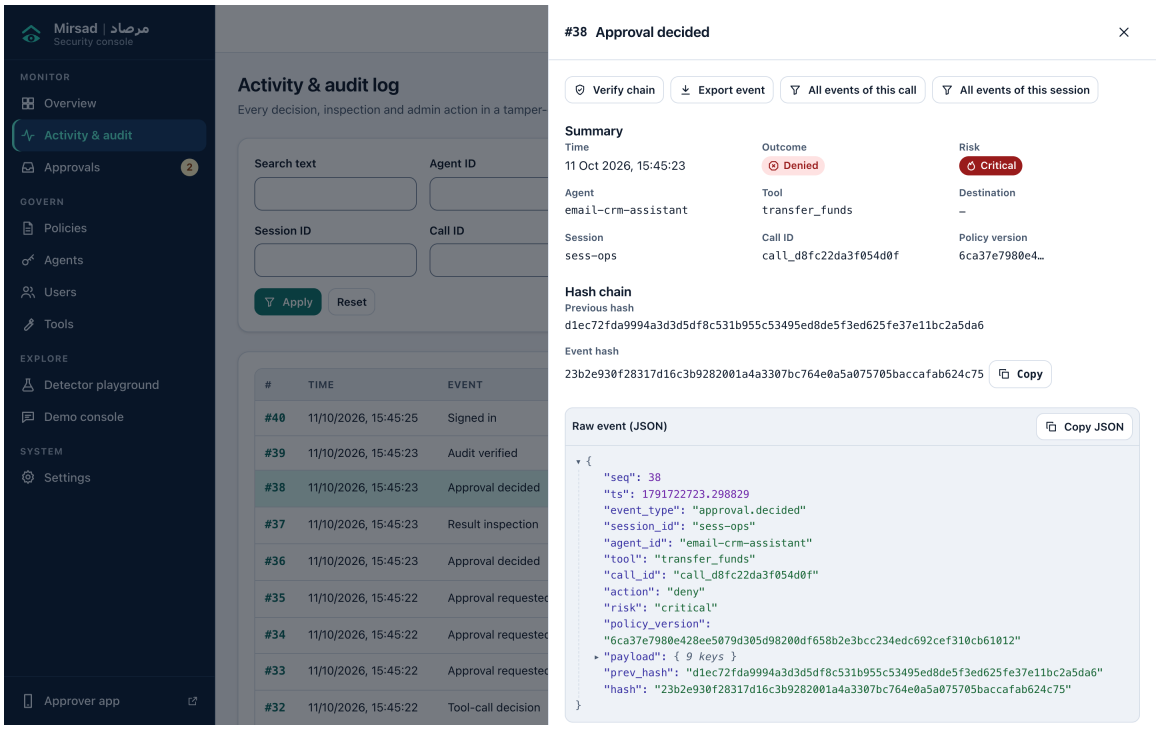


Figure 2: Activity and audit log with the detail drawer of an approval decision

E.5.4 Approvals

The pending queue (Figure 3) shows paused calls, soonest expiry first, with risk, tool, agent, countdown and a plain-language summary. **Review** opens the full explanation (E.6.2); **Approve** and **Deny** ask for confirmation and an optional note. If another approver decided first, the page says so. The history lists past requests with the decision maker and note.

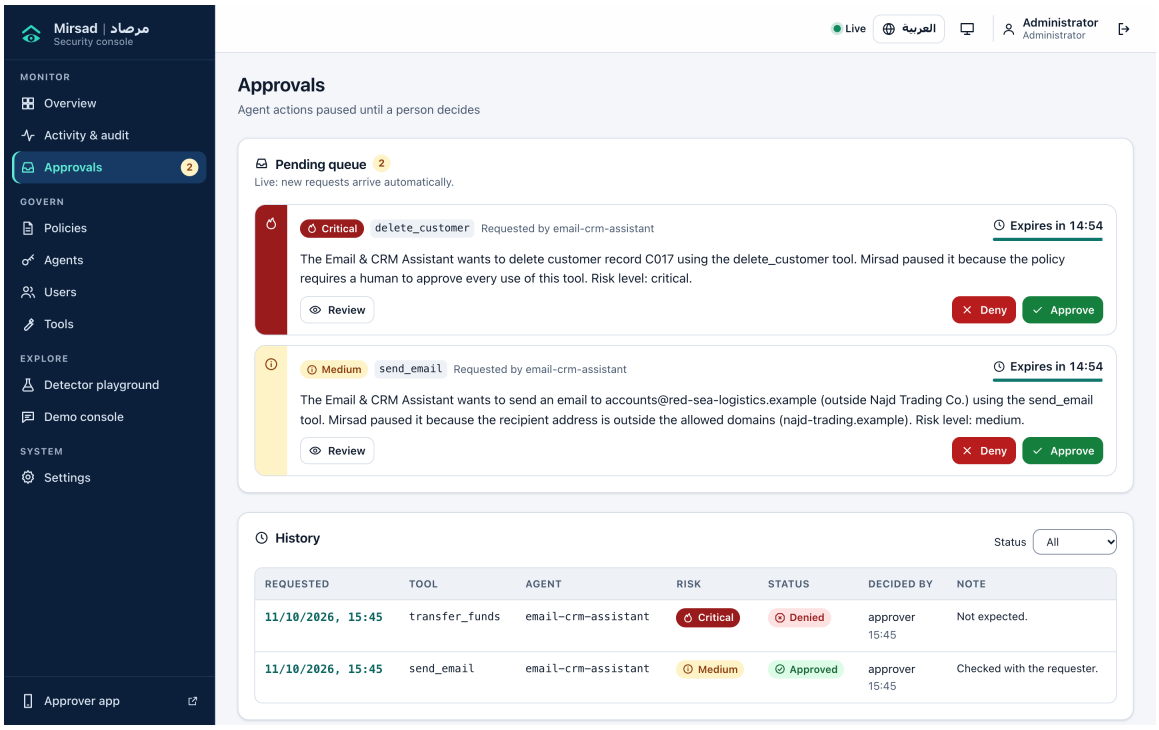


Figure 3: Approvals page with two pending requests and the decision history

E.5.5 Policies, agents and users

Policies (Figure 4) shows the active YAML in a line-numbered editor with **Validate** (errors link to their line), **Compare with active**, **Activate** (diff and change note) and a version history from which administrators can re-activate an older version. Auditors get a read-only view; E.7 explains policy writing.

Agents (Figure 5) lists agent identities with policy agent, key prefix, last use and status. A new or rotated key appears once, in a dialog that closes only after "I have stored this key" is ticked, together with any warning, for example a policy agent id missing from the active policy. Rotation invalidates the old key immediately; **Disable** blocks an agent but keeps its history. **Users** creates accounts with the roles `admin`, `approver` or `auditor`, by default with a server-generated password shown once. The dashboard does not let you change your own role or status, and the last active administrator cannot be demoted, disabled or deleted.

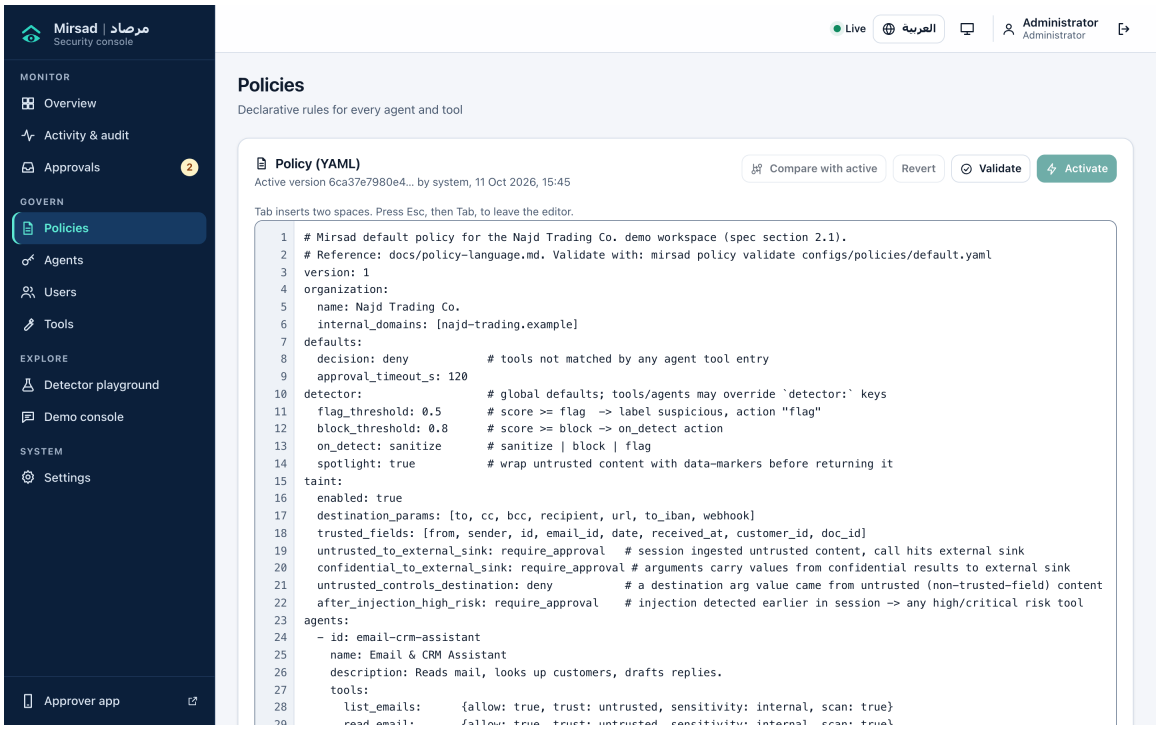


Figure 4: Policies page with the YAML editor

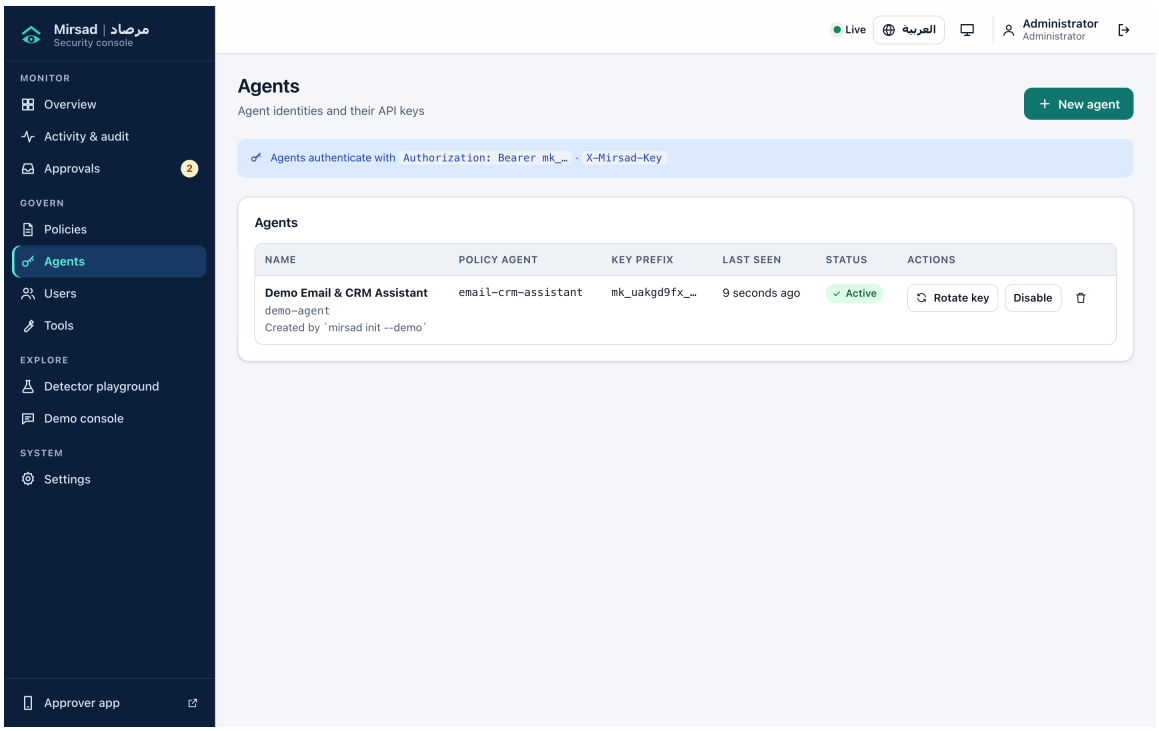


Figure 5: Agents page with the demo agent created by mirsad init --demo

E.5.6 Tools, detector playground, demo console and settings

Tools shows each discovered upstream tool and how the active policy treats it per agent (allowed, trust, sensitivity, sink, risk, approval, scanning). The **Detector playground**

(Figure 6) scans pasted text and shows score, thresholds, verdict, language, model and rule scores, latency, highlighted evidence and obfuscation signals; teams can add labelled samples in `src/mirsad/static/assets/playground-samples.json`. The **Demo console** (Figure 7) holds the protection switch, scenario selector, conversation, tool-call timeline, attacker inbox and a count of workspace changes. The attacker inbox lists every message that left the organisation, including legitimate ones; the item in Figure 7 is an approved reply to a supplier. The page never displays the planted content itself. **Settings** covers theme, language, Western or Arabic-Indic digits, desktop alerts and the server version.

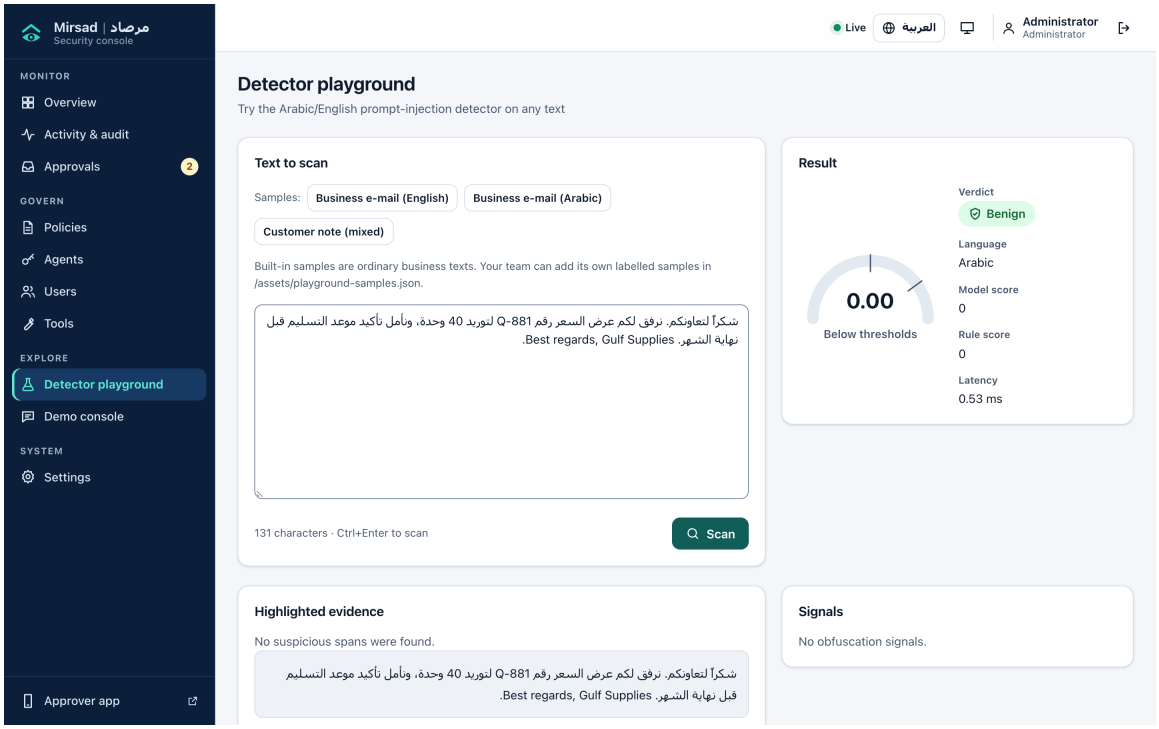


Figure 6: Detector playground scanning an Arabic business e-mail

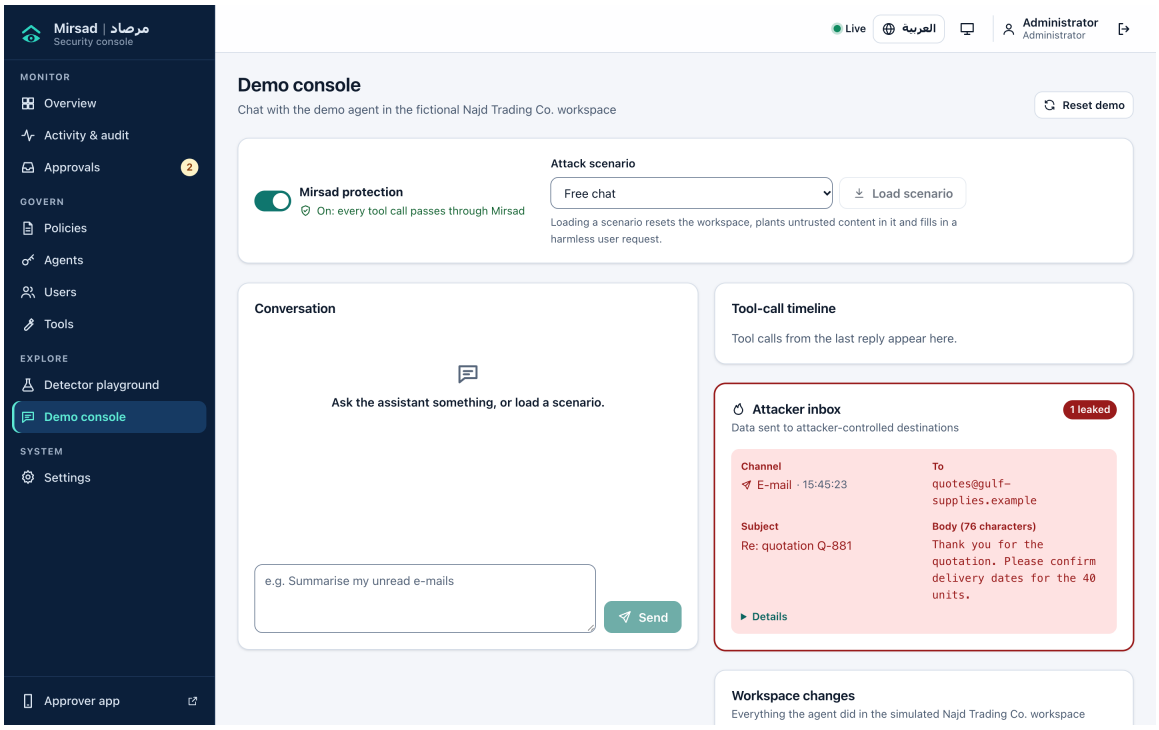


Figure 7: Demo console with the protection switch, scenario selector and attacker inbox

E.6 Approver App

The approver app is an installable progressive web app [5], [6] at `/approve`. Browsers allow installation, service workers and push only on secure origins, so the server must use HTTPS (E.13); `http://localhost` works only on the same machine.

E.6.1 Installing on Android and iOS

Android (Chrome, Edge or Samsung Internet):

1. Open `https://<your-mirsad-host>/approve` and sign in with an approver account.
2. Tap **Install** on the "Install Mirsad Approver" card, or choose **Install app** / **Add to Home screen** in the browser menu.
3. Open Mirsad from the home screen, tap **Turn on alerts** and allow notifications.

iPhone or iPad (Safari; alerts need iOS/iPadOS 16.4 or later):

1. Open the same address in Safari and sign in.
2. Tap **Share**, **Add to Home Screen**, **Add**.
3. Open Mirsad from the home screen (not Safari), tap **Turn on alerts** and allow. iOS delivers web notifications only to home-screen apps [7].

The sign-in ends when the app is fully closed, by design. The phone screenshots were taken in Chrome emulating a 390 × 844 screen; installation and push delivery on physical phones were not part of the recorded tests.

E.6.2 Reviewing and deciding

Pending (Figure 8) shows one card per request with risk, countdown, tool, agent and summary. The list updates live, or every 20 s while the live connection is down. The detail view (Figure 9) has three parts:

- **What the agent wants to do**: a plain-language summary.
- **Why Mirsad paused it**: only the reasons that caused the pause, each with its layer.
- **Where the data came from**: the source tool, call and field of each value that came from outside content.

The arguments follow, collapsed. **Approve** and **Deny** open a sheet that restates the consequence and accepts a note, which is audited. Both are disabled when the request has expired or the phone is offline. A request not decided in time is denied, and the waiting agent receives the denial. **History** lists past requests.

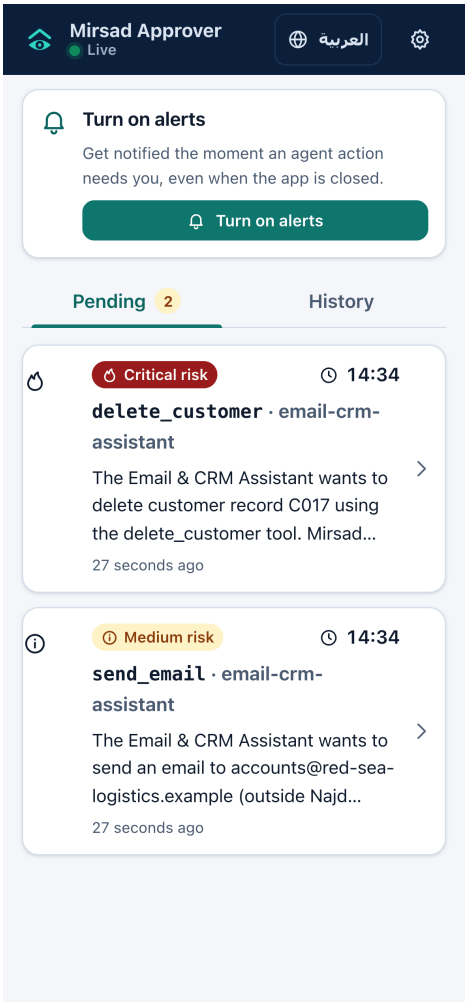


Figure 8: Approver app: pending requests

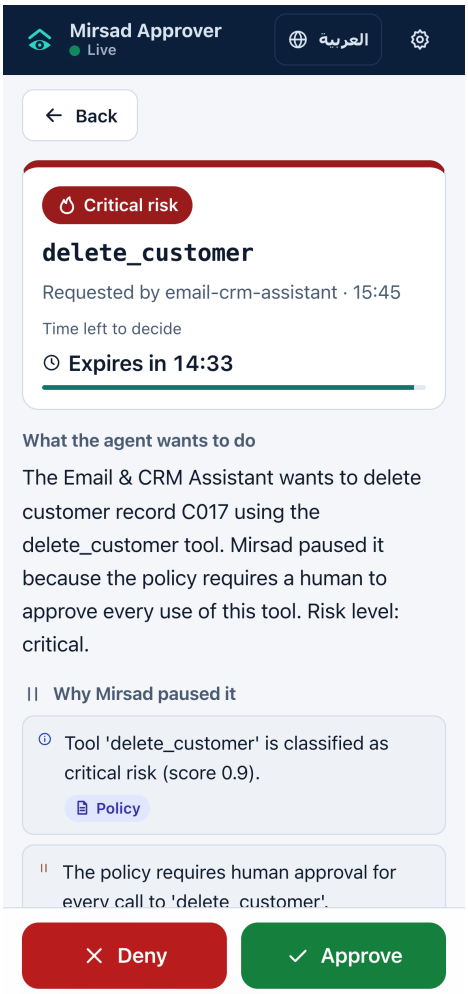


Figure 9: Approver app: request detail with the Approve and Deny buttons

E.6.3 Notifications

While the app runs in the background it shows a system notification for each new request. For alerts while it is closed, the administrator enables Web Push with a VAPID key pair and restarts the server:

```
python -c "from mirsad.approvals import generate_vapid_keys; print(*generate_vapid_keys(), sep='\n')"
# line 1 -> MIRSAD_VAPID_PUBLIC_KEY, line 2 -> MIRSAD_VAPID_PRIVATE_KEY
```

Push messages pass, encrypted, through the browser vendor's push service; leave the keys unset if all data must stay on premises. Signing out removes the device's subscription. Push delivery was tested only against a simulated push library, so test it on your own phones first.

E.6.4 Arabic interface

The globe button switches to Arabic, which mirrors the layout right-to-left (Figure 10); summaries and reasons arrive from the server in Arabic, while tool names, addresses and arguments stay left-to-right. A short instruction for Arabic-speaking approvers:

افتح تطبيق «معتمِد مرصاد» من الشاشة الرئيسية، واضغط على الطلب لتقرأ ما يريد الوكيل فعله، ولماذا أوقفه مرصاد، ومن أين جاءت البيانات. ثم اختر «موافقة» أو «رفض» مع ملاحظة عند الحاجة. إذا انتهت المهلة دون قرار يُرفض الطلب تلقائياً.

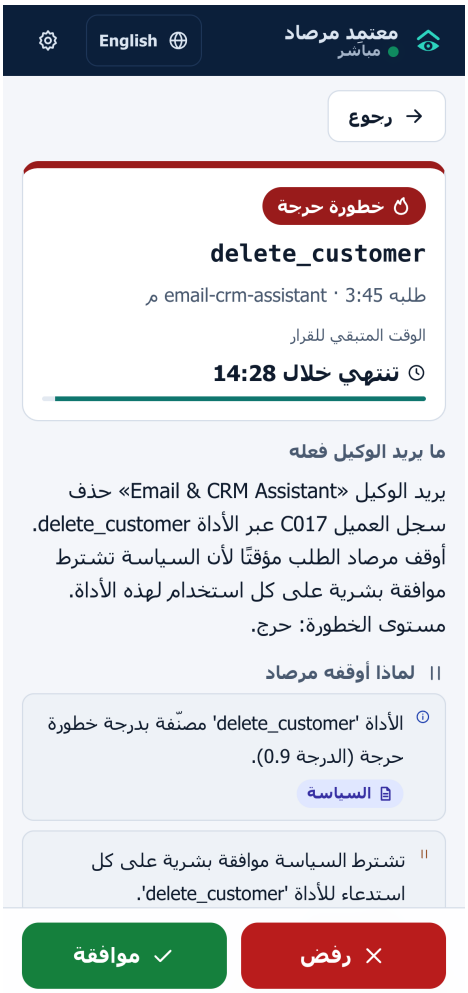


Figure 10: Approver app: request detail in Arabic

E.7 Writing and Activating Policies

E.7.1 Policy structure and an example

A policy is one YAML document with `version`, `mode` (`enforce` or `monitor`), `organization.internal_domains`, `defaults`, `detector`, `taint`, `agents` and `rules`. Each agent lists its tools with `allow`, result trust and sensitivity, sink, risk, approval mode, argument constraints and rate limits. The agent `id` must equal the policy agent of the agent's key.

The full reference is `docs/policy-language.md`; samples are in `configs/policies/`. This example passed `mirsad policy validate`:

```
version: 1
mode: monitor                       # shadow mode first; change to enforce after tuning
organization:
  name: Najd Trading Co.
  internal_domains: [najd-trading.example]
defaults: {decision: deny, approval_timeout_s: 120}
agents:
  - id: support-assistant
    name: Customer Support Assistant
    tools:
      read_email: {allow: true, trust: untrusted, scan: true}
      get_customer: {allow: true, sensitivity: confidential, scan: true}
      send_email:
        allow: true
        sink: dynamic
        sink_params: [to, cc, bcc]
        risk: medium
        constraints:
          to: {domain_in: [najd-trading.example], on_violation: require_approval}
        rate_limit: {max_calls: 20, per_seconds: 3600}
      export_customers:
        allow: true
        sensitivity: confidential
        risk: high
        approval: on_risk
        constraints:
          count: {max: 50, on_violation: deny}
      "*": {allow: false}
```

Evaluating sample calls with the policy engine gave these decisions, which the gateway enforces once `mode` is `enforce`: an internal e-mail is allowed; an e-mail to an outside domain is paused (constraint violation); an export of 200 records is denied; an unlisted tool is denied (`TOOL_NOT_ALLOWED`). A smaller export is paused, because a `high` risk tool under `approval: on_risk` always reaches the approval threshold. Reading e-mail marks the session as tainted, so the default taint rules also apply: a later outside e-mail adds `TAINT_UNTRUSTED_TO_EXTERNAL`, and a recipient copied from an e-mail body is denied.

E.7.2 Validating

Run `mirsad policy validate FILE` (exit code 1 when invalid) locally or in continuous integration. Errors give line, column, path and a suggestion; two typing mistakes in the example produce:

```
error: line 16, column 9: agents[0].tools.send_email.sink_param: unknown key 'sink_param'
(did you mean 'sink_params'?); ...
error: line 24, column 15: agents[0].tools.export_customers.risk: invalid value 'hgih': ...
(did you mean 'high'?)
```

The validator also rejects duplicate keys, regular expressions prone to catastrophic backtracking, ambiguous tool globs and inconsistent thresholds.

E.7.3 Activating, monitor mode and rollback

Activate a policy with **Activate** on the Policies page, with `mirsad policy activate FILE --comment "..."` (a running server picks it up within about 5 s) or with `PUT /api/policies`. Activation needs no restart, keeps rate-limit counters and is audited as `policy.activated`. A version is the SHA-256 hash of the normalised document, so comment-only edits keep the version.

In **monitor mode** the gateway audits what it would have decided but always allows, never pauses and never alters results; such events are tagged "monitor mode: not enforced". A safe rollout is: activate with `mode: monitor`, review the would-be denials and approvals during normal work, adjust, then activate with `mode: enforce`. To **roll back**, re-activate an earlier version from the history, call `POST /api/policies/{version}/activate`, or activate the old file again (same content, same version).

E.8 Connecting Agents

Mirsad protects only traffic that passes through it. All paths in Table 5 use the same pipeline and audit log.

Table 5: Ways to connect an agent

Agent	Mirsad entry point	Agent-side change
MCP host launching local servers (Claude Desktop, IDEs)	`mirsad mcp-proxy` (stdio)	Replace tool servers with the proxy
MCP client with remote servers	`/mcp` (streamable HTTP)	URL and authorisation header
OpenAI-compatible client	`/v1/chat/completions`	`base_url` and `api_key`
Python code running its own tools	SDK `@guard` decorator	Decorate tool functions
Any language	`/v1/guard/tool-call`, `/v1/guard/tool-result`	One call before and after each tool

E.8.1 Before connecting

1. Create an agent in the dashboard or with `mirsad agent create "Support Assistant" --policy-agent support-assistant --key-file support.key`. The key (`mk_<prefix>_<secret>`) is shown or written once.
2. Make sure the active policy defines that policy agent; otherwise every call is denied with `UNKNOWN_AGENT`.
3. On the MCP paths Mirsad runs the tools itself: declare the tool servers in `configs/upstreams.yaml` (`inprocess`, `stdio` or `http`; see `docs/integration.md`). The shipped file connects only the demo workspace.
4. Remove the agent's direct access to the original tool servers, or it can bypass Mirsad.

5. Use one session id per conversation (`X-Mirsad-Session`, `--session-id` or the SDK's `session_id`); taint tracking works per session.

E.8.2 MCP hosts: stdio proxy and HTTP

`mirsad mcp-proxy` is a stdio MCP server that forwards each call to a running Mirsad server. For Claude Desktop, in `claude_desktop_config.json` (macOS: `~/Library/Application Support/Claude/`):

```
{
  "mcpServers": {
    "mirsad": {
      "command": "/opt/mirsad/.venv/bin/mirsad",
      "args": ["mcp-proxy", "--url", "https://mirsad.najd-trading.example"],
      "env": { "MIRSAD_AGENT_KEY": "mk_xxxxxxxx_yyyyyyyyyyyyyyyy" }
    }
  }
}
```

Hosts that support remote servers connect instead to `https://<host>/mcp` with the header `Authorization: Bearer mk_...`; the configuration format depends on the host. Either way the host sees only the tools its policy allows, and a denied call returns an MCP tool error explaining the block in English and Arabic.

E.8.3 OpenAI-compatible agents

```
from openai import OpenAI

client = OpenAI(base_url="https://mirsad.najd-trading.example/v1",
                api_key="mk_xxxxxxxx_yyyyyyyyyyyyyyyy",
                default_headers={"X-Mirsad-Session": "conversation-42"},
                timeout=180)   # longer than the approval timeout
```

Mirsad inspects every `role: tool` message before forwarding the request to `MIRSAD_LLM_BASE_URL` (never with the agent key), removes tools the policy forbids, and checks each tool call in the reply, removing denied calls and appending a bilingual explanation. Streaming is not supported (HTTP 400). Here the agent's own code runs the tools, so Mirsad assumes that code is honest; prefer the MCP paths when it is not.

E.8.4 Python SDK

```
from mirsad.sdk import MirsadClient, MirsadBlocked

mirsad = MirsadClient("https://mirsad.najd-trading.example", api_key="mk_...",
                      session_id="conversation-42")

@mirsad.guard(name="send_email")          # policy tool name
def send(to: str, subject: str, body: str) -> str:
    return smtp_send(to, subject, body)    # existing tool code

try:
    text_for_model = send("ops@najd-trading.example", "Stock report", "...")
except MirsadBlocked as exc:
    text_for_model = str(exc)              # bilingual explanation for the model
```

The decorator checks the call, runs the function only if allowed, sends the result for inspection and returns the inspected text; always give the model that text, not the raw value. `MirsadClient.from_env()` reads `MIRSAD_URL` and `MIRSAD_AGENT_KEY`; the default timeout of 150 s covers an approval wait.

E.9 Command-Line Reference

Usage is `mirsad [--data-dir DIR] [--log-level LEVEL] COMMAND`; results go to standard output and logs to standard error. Table 6 was compiled from the `--help` output of version 1.0.0.

Table 6: Commands of the mirsad command line

Command	Purpose
`serve [--host] [--port] [--demo/--no-demo]`	Run the gateway, API and web apps
`init [--demo] [--admin-password PW]`	Create the database, first policy version, users (and demo agent)
`policy validate FILE`	Check a policy; exit code 1 when invalid
`policy activate FILE [--comment TEXT]`	Store and activate a policy version
`audit verify`	Recompute the hash chain; exit code 1 if broken
`audit export [--format json|csv] [--out FILE] [--agent-id] [--tool] [--action] [--event-type] [--since] [--until]`	Export events (times in epoch seconds)
`audit replay SEQ`	Re-evaluate a stored decision or inspection
`scan TEXT` or `scan --file FILE`	Run the injection detector
`agent create NAME --policy-agent ID [--id] [--description] [--key-file FILE]`	Register an agent; key printed once or written to a mode-600 file
`user create USERNAME --role admin|approver|auditor [--password] [--display-name] [--lang en|ar]`	Create a user (password prompted if omitted)
`mcp-proxy --agent-key KEY [--url URL] [--session-id ID]`	Stdio MCP server forwarding to a running gateway
`detector train [ARGS]`	Train the detector (`python -m mirsad.detector.train`)

The Makefile wraps common tasks: `make setup`, `serve`, `demo`, `web`, `test` (without model-dependent tests), `test-all`, `lint`, `security` (Bandit, pip-audit, npm audit), `bench`, `latency`, `detector` and `docs`.

E.10 Audit Verification, Replay and Export

Decisions, inspections, approvals, policy activations, failed sign-ins and user or agent changes are audit events. Each event's hash is an HMAC-SHA-256 over its content and the previous hash, keyed from `MIRSAD_SECRET_KEY` [1], [8]; secrets are removed and identifiers such as IBANs and national IDs masked before hashing.

Verify with `mirsad audit verify`, **Verify chain** in the dashboard or `POST /api/audit/verify`. The report gives `ok`, `checked`, `head_seq`, `head_hash` and, on failure, `first_bad_seq` with the reason (modified event, missing sequence number, broken link or wrong key). The command exits with code 1 on failure, so it can be scheduled.

Replay (`mirsad audit replay SEQ` or **Replay**) re-evaluates a `tool_call.decision` or `tool_result.inspection` event with the policy version it recorded and reports whether the action matches. Masked or truncated arguments can cause differences, which the report notes; inspections are replayed with the current detector model. In the scripted benchmark 272 of 272 audited decisions replayed to the same action.

Export with `mirsad audit export --format json --out audit.json`. JSON keeps the chain fields and is the evidence format; CSV is for reading, with formula-like cells escaped. An unfiltered JSON export verifies offline with the key (filtered exports have gaps):

```
import json
from mirsad.audit import audit_key_from_secret, verify_events

export = json.load(open("audit.json", encoding="utf-8"))
key = audit_key_from_secret(open(".mirsad/secret.key").read().strip())  # or the
MIRSAD_SECRET_KEY value
anchor = {"seq": 4, "hash": "<head_hash recorded earlier>"}             # optional, see E.13
print(verify_events(export["events"], key, anchor=anchor))
```

A valid chain shows that nobody without the key changed, removed or reordered events inside it. Deleting the newest events, or a rewrite by a key holder, is detected only against a head recorded earlier outside the database (an anchor); version 1.0 checks anchors in Python, not from the command line.

E.11 Training the Detector and Running the Benchmark

E.11.1 Detector

Mirsad ships without a trained model, so the detector runs in rules-only mode. The attack-labelled data is written and reviewed by the project team (format in `docs/detector.md`, sections 11–12), not by the AI assistant used during development; `data/detector/raw/` currently holds only benign batches. To train a model:

1. Add training batches as `data/detector/raw/<batch>.jsonl` and separately written test batches as `raw/test-<name>.jsonl`.
2. Run `make detector`: it builds the splits (deduplication, split by batch, training-only augmentation, leakage check), trains, selects thresholds on the validation split and writes `src/mirsad/detector/model/`, `reports/detector_metrics.json` and figures.
3. Check the test metrics, run `pytest tests/detector`, and restart the server to load the model.

E.11.2 Benchmark

`make bench` runs 48 benign English and Arabic user tasks under every defence configuration, with an approver that denies every escalation and one that accepts only what the user asked for, and writes `reports/bench/comparison.md` (other runs: `docs/benchmark.md`). `scripts/run_evidence.sh` re-runs tests, scans and benchmarks, `scripts/collect_results.py` refreshes `reports/results_summary.json`, and `make docs` rebuilds this document.

Attack files are supplied by the team in `bench/scenarios/attacks/*.yaml` (format in the README of that folder) and checked with `python -m bench.loader`. Once they exist, `make bench` (`scripts/run_bench.sh`) also runs the attack episodes, and the attack success rates fill in.

Table 7: Headline results and how they are produced

Result	Value	Produced by
Benign false-positive rate, full protection	0.0%	`make bench`
Approval friction on benign calls, full protection	17.6%	`make bench`
Utility with the plan-aware approver, full protection	100.0%	`make bench`
Added latency, sequential median	2.7 ms	`python -m bench.latency --server-process`
Detector F1, English test set	[pending]	`make detector` with team data
Detector F1, Arabic test set	[pending]	`make detector` with team data
Attack success rate reduction, full vs none	[pending]	Attack episodes with team attack files

E.12 Troubleshooting and FAQ

Table 8: Common problems and solutions

Symptom	Likely cause	Action
Agent gets 401	Wrong, rotated or disabled key	Check the key prefix on the Agents page
All calls denied with `UNKNOWN_AGENT`	Policy agent not in the active policy	Add the agent to the policy
Tool missing or `TOOL_NOT_ALLOWED`	No `allow: true` entry for the tool	Add it to the agent's tools
503 on `/v1/tools/.../invoke`	No upstream tool servers	Check `MIRSAD_UPSTREAMS_PATH` and the log
Call hangs about two minutes, then denied	Approval expired	Enable approver alerts; raise client timeouts
Demo Console error 502	Ollama or model missing	`ollama pull qwen2.5:7b`
"Too many attempts" at sign-in	Login throttling	Wait 5 minutes

Symptom	Likely cause	Action
`/` shows a JSON note	Web app not built	`make web`
No push alerts	No HTTPS, VAPID keys or permission, or iOS app not on home screen	See E.6.1 and E.6.3
Nothing is blocked	Policy in `mode: monitor`	Activate with `mode: enforce`
`audit verify` fails at sequence 1	`MIRSAD_SECRET_KEY` differs from the one that wrote the log	Restore the original key
OpenAI client gets 400	`stream: true`	Use non-streaming requests
`make detector` stops: "train split needs both benign (0) and injection (1) examples"	Only benign batches in `data/detector/raw/` (the current state)	Add the team's labelled batches (E.11.1)

Can I run several Mirsad processes behind a load balancer? Not in version 1.0: taint sessions, rate-limit counters and the login throttle live in memory per process.

Does Mirsad send data elsewhere? Only to what you configure: upstream tool servers, the upstream LLM and, with VAPID keys, browser push services.

Does Mirsad stop a user who types a jailbreak? No. It governs tool calls and results; a harmful tool call is still subject to the policy whatever caused it.

E.13 Security Recommendations for Operators

Secret key. Supply `MIRSAD_SECRET_KEY` from a secrets manager (for example `python -c "import secrets; print(secrets.token_urlsafe(48))"`) rather than the generated `secret.key`, which sits beside the database and its backups. Restrict the data directory with `chmod 700`: on the development machine the SQLite file was created with mode 644, while `secret.key` and `credentials.txt` were mode 600.

Changing the secret key. It signs sign-in tokens and is the source of the audit HMAC key, so changing it signs everyone out and makes `audit verify` fail at sequence 1. Version 1.0 cannot re-key the chain. Change it only after a suspected compromise, and first verify, export the log as JSON, record the head and archive the old key separately so the old history stays verifiable offline (E.10).

Agent keys and passwords. Give each agent its own key and rotate keys on a schedule, when staff leave and after any suspected leak. Keep passwords and keys in a password manager, delete `credentials.txt`, and assign the least powerful role that fits.

TLS reverse proxy. Keep Mirsad on `127.0.0.1` behind a proxy that terminates TLS with a valid certificate, for example from Let's Encrypt [9]. Configure the proxy to send `Strict-Transport-Security` (Mirsad adds it only when it receives HTTPS itself), to pass `/api/stream` unbuffered, to allow read timeouts longer than the approval timeout on `/v1/*` and `/mcp`, and to restrict those paths to agent networks. Mirsad does not trust forwarded-

address headers, so behind a proxy the per-address login throttle is shared by all clients. With Compose, publish port 8700 only to the proxy.

Configuration. Set `MIRSAD_ENABLE_DEMO=false`, set `MIRSAD_CORS_ORIGINS` to `[]` or the dashboard origin, keep `defaults.decision: deny`, introduce policies in monitor mode, and remove direct routes from agents to tool servers.

Anchoring audit heads. At least daily and after each verification, record `head_seq` and `head_hash` from `mirsad audit verify` somewhere database administrators cannot change, such as an e-mail to auditors or write-once storage [10], and verify exports against these anchors (E.10).

Backups and updates. Back up the database and keep the key elsewhere. Run `make security` before upgrades; for this release Bandit reported 0 issues, pip-audit 0 vulnerable packages among 124, and npm audit 0 vulnerabilities.

References

[1] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997, doi: 10.17487/RFC2104.

[2] Model Context Protocol, "Specification," protocol revision 2026-07-28. Accessed: Oct. 11, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28.

[3] Ollama, "OpenAI compatibility," Ollama Docs. [Online]. Available: https://docs.ollama.com/api/openai-compatibility. Accessed: Oct. 11, 2026.

[4] Qwen Team, "Qwen2.5-7B-Instruct," model card, Hugging Face. Accessed: Oct. 11, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-7B-Instruct.

[5] M. Cáceres, D. Murphy, and C. Liebel, Eds., "Web Application Manifest," W3C Working Draft, Oct. 9, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/appmanifest/

[6] M. Chintala and Y. Yanagisawa, Eds., "Service Workers Nightly," W3C Candidate Recommendation Draft, Sep. 17, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/service-workers/

[7] B. Eidson and J. Simmons, "Web Push for Web Apps on iOS and iPadOS," WebKit Blog, Feb. 16, 2023. [Online]. Available: https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/. Accessed: Oct. 11, 2026.

[8] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in Proc. 18th USENIX Secur. Symp., Montreal, QC, Canada, Aug. 2009.

[9] Internet Security Research Group, "About Let's Encrypt," Let's Encrypt. [Online]. Available: https://letsencrypt.org/about/. Accessed: Oct. 11, 2026.

[10] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," J. Cryptol., vol. 3, no. 2, pp. 99–111, 1991, doi: 10.1007/BF00196791.